

PECS-PRO v2 — E15

Migration Rationale + Full Module Responsibility Reference

SECTION 1 — WHY PECS MOVED FROM v1 TO v2

1.1 The Original PECS Problem

PECS originally existed to solve continuity fragmentation in AI-assisted engineering.

Modern LLM-assisted engineering workflows suffer from:

- continuity drift
- duplicate evolution
- topology blindness
- hallucinated locality
- fragmented ownership
- repeated rediscovery
- loss of execution continuity
- exploding context windows
- inability of small-context models to survive large projects

The original PECS idea attempted to preserve:

- code continuity
- ownership continuity
- implementation locality
- engineering memory
- duplicate tracking

through:

- file indexing
- imports
- dependency scanning
- duplicate detection
- static continuity mapping

This became PECS v1.

SECTION 2 — WHY PECS v1 BECAME INSUFFICIENT

2.1 The Core Architectural Failure

PECS v1 was still fundamentally:

file-hierarchical.

It modeled:

- files
- imports
- dependencies
- duplicate files
- static continuity

But modern GUI systems — especially Qt systems — do NOT derive continuity primarily from file relationships.

They derive continuity from:

- runtime composition
- signal-slot chains
- QAction registration
- dispatch propagation
- overlay propagation
- subprocess orchestration
- runtime ownership
- execution paths
- viewer propagation
- callback chains

Meaning:

continuity in complex engineering systems is execution-topological.

NOT:

file-hierarchical.

This became the foundational realization behind PECS v2.

SECTION 3 — SPECIFIC PECS v1 FAILURE MODES

3.1 Runtime Blindness

PECS v1 could not correctly reconstruct:

- runtime dispatch
- signal-slot continuity
- QAction ownership
- runtime execution locality
- overlay propagation
- viewer ownership
- callback topology
- subprocess execution continuity

Meaning:

PECS v1 frequently reconstructed incorrect locality.

3.2 Context Explosion

PECS v1 increasingly accumulated:

- giant continuity dumps
- duplicate indexing layers
- overlapping retrieval paths
- repeated continuity structures
- redundant orchestration logic

This caused:

- token inflation
- retrieval entropy
- locality fragmentation
- excessive traversal depth

which made PECS itself continuity-fragmented.

3.3 Middleware Proliferation

PECS v1 gradually evolved into:

- bridges
- middleware
- coordinators
- orchestrators
- sync layers

- runtime daemons
- injection systems
- wrappers around wrappers

Examples:

- multiple retrieval coordinators
- multiple injection systems
- multiple runtime bridges
- multiple export paths
- duplicate verification systems
- duplicate continuity ownership

This created:

authority duplication.

Authority duplication became the single largest PECS v1 architectural failure.

3.4 Continuity Fragmentation Inside PECS

The irony of PECS v1 was:

PECS itself became continuity-fragmented.

The architecture drifted toward:

feature-driven decomposition

instead of:

authority-driven decomposition.

That caused:

- overlapping responsibilities
- unclear ownership
- duplicate validation paths
- duplicate consolidation layers
- duplicate retrieval flows
- duplicate runtime authorities

which eventually became unsustainable.

SECTION 4 — THE CORE PECS v2 BREAKTHROUGH

The critical realization became:

continuity is execution-topological.

NOT:

file-hierarchical.

This changed the architecture completely.

PECS v2 now reconstructs:

- execution locality
- runtime topology
- dispatch continuity
- signal-slot continuity
- ownership continuity
- propagation continuity
- compact continuity reconstruction

rather than:

- giant static dependency memory.

SECTION 5 — THE NEW PECS v2 PHILOSOPHY

PECS v2 intentionally avoids:

- AGI orchestration
- recursive AI systems
- autonomous engineering systems
- semantic orchestration forests
- self-modifying infrastructure
- middleware-heavy architecture
- giant persistent conversational memory

PECS v2 instead focuses on:

- deterministic continuity
 - topology-first reconstruction
 - execution-local retrieval
 - continuity compression
 - locality preservation
 - low-token reconstruction
 - continuity density
-

SECTION 6 — THE MOST IMPORTANT PECS v2 PRINCIPLE

ONE authority module per continuity concern.

This became the anti-fragmentation rule.

Meaning:

If a module already owns:

- retrieval
- scoring
- incremental rebuilding
- runtime continuity
- validation
- export logic

then no new module may partially re-own those responsibilities.

This prevented:

- duplicate validation paths
- duplicate verification systems
- duplicate consolidators
- duplicate retrieval coordinators
- orchestration proliferation

This became one of the most important architectural corrections from PECS v1.

SECTION 7 — WHAT PECS v2 ACTUALLY BECAME

PECS-PRO v2 became:

compressed topology-first continuity infrastructure.

Its purpose is now:

minimal-context topology-directed continuity reconstruction.

Meaning:

Given:

- an object
- its role

- its execution locality
- expected outcome
- runtime topology

PECS should help an LLM directly reconstruct:

- where to look
- what owns the execution
- what locality matters
- what propagation chain matters
- what dispatch chain matters

WITHOUT:

- scanning entire workspaces
- rereading giant files
- retaining massive chat histories
- repeatedly rediscovering continuity

This is the actual PECS v2 objective.

SECTION 8 — FINAL PECS v2 ARCHITECTURE

Canonical structure:

```
pecs_pro/  
  continuity/  
  runtime/  
  execution_graph/  
  topology/  
  integrations/  
  exports/  
  validation/  
  run_pecs_pro.py
```

The architecture is now:

- topology-first
 - continuity-oriented
 - deterministic
 - dense-authority
 - low-orchestration
 - low-token
 - migration-safe
-

SECTION 9 — FULL MODULE RESPONSIBILITY REFERENCE

CONTINUITY LAYER

Purpose

The continuity layer defines:

- canonical continuity primitives
- continuity ownership structures
- continuity snapshots
- duplicate tracking
- runtime continuity state

This layer is foundational.

It represents:

the core continuity object model.

runtime_node.py

Purpose:

Canonical runtime continuity node.

Represents:

- runtime entities
- execution-local objects
- continuity anchors
- ownership surfaces

Effect:

Provides the foundational runtime continuity primitive used by:

- topology reconstruction
 - execution graphs
 - locality indexing
 - retrieval
 - continuity reconstruction
-

execution_edge.py

Purpose:

Canonical execution continuity edge.

Represents:

- execution flow
- execution transitions
- runtime adjacency
- execution locality relationships

Effect:

Provides deterministic execution continuity relationships.

ownership_edge.py

Purpose:

Represents ownership continuity.

Tracks:

- object ownership
- runtime ownership
- viewer ownership
- execution ownership

Effect:

Stabilizes ownership locality reconstruction.

dispatch_edge.py

Purpose:

Represents runtime dispatch continuity.

Tracks:

- dispatch propagation
- runtime routing
- dispatch adjacency

Effect:

Preserves dispatch-local continuity.

propagation_edge.py

Purpose:

Represents propagation continuity.

Tracks:

- overlay propagation
- state propagation
- runtime propagation chains

Effect:

Stabilizes propagation-local execution continuity.

subprocess_edge.py

Purpose:

Represents subprocess execution continuity.

Tracks:

- subprocess calls
- execution-chain continuity
- external runtime execution

Effect:

Preserves subprocess topology reconstruction.

continuity_zone.py

Purpose:

Defines continuity-local zones.

Represents:

- locality clusters
- continuity surfaces
- execution-local engineering regions

Effect:

Allows PECS to reconstruct engineering locality.

continuity_cluster.py

Purpose:

Represents grouped continuity structures.

Tracks:

- continuity relationships
- locality groups
- duplicate evolution clusters

Effect:

Provides structured continuity grouping.

continuity_snapshot.py

Purpose:

Stores continuity state snapshots.

Effect:

Allows deterministic continuity preservation.

topology_snapshot.py

Purpose:

Stores topology reconstruction snapshots.

Effect:

Preserves topology state continuity.

execution_snapshot.py

Purpose:

Stores execution continuity snapshots.

Effect:

Preserves execution-local continuity state.

REGISTRIES

continuity_registry.py

Purpose:

Canonical continuity registration authority.

Tracks:

- continuity objects
- continuity relationships
- canonical continuity state

Effect:

Provides deterministic continuity registration.

runtime_registry.py

Purpose:

Canonical runtime registration authority.

Tracks:

- runtime entities
- runtime topology participants

Effect:

Provides runtime continuity authority.

ownership_registry.py

Purpose:

Tracks ownership relationships.

Effect:

Provides canonical ownership locality authority.

duplicate_registry.py

Purpose:

Tracks duplicate evolution.

Effect:

Allows continuity archaeology and duplicate drift tracking.

CONFIDENCE LAYER

Purpose

The confidence layer determines:

- runtime authority weighting
- topology confidence
- ownership confidence
- dispatch confidence
- continuity confidence

This layer remains:

observational only.

It NEVER mutates runtime truth.

confidence_model.py

Purpose:

Canonical confidence model.

Effect:

Provides unified confidence representation.

runtime_confidence.py

Purpose:

Scores runtime-verified continuity.

Effect:

Increases authority of runtime-confirmed topology.

topology_confidence.py

Purpose:

Scores topology reconstruction confidence.

Effect:

Weights topology continuity reliability.

ownership_confidence.py

Purpose:

Scores ownership locality confidence.

Effect:

Improves ownership continuity stability.

dispatch_confidence.py

Purpose:

Scores dispatch continuity confidence.

Effect:

Improves dispatch-local retrieval.

propagation_confidence.py

Purpose:

Scores propagation continuity.

Effect:

Improves propagation-local reconstruction.

subprocess_confidence.py

Purpose:

Scores subprocess continuity confidence.

Effect:

Improves subprocess topology stability.

canonical_confidence.py

Purpose:

Produces canonical continuity confidence.

Effect:

Provides unified continuity authority weighting.

RUNTIME LAYER

Purpose

The runtime layer reconstructs:

- runtime topology
- execution locality
- signal-slot continuity
- QAction continuity
- dispatch continuity
- subprocess topology
- overlay propagation
- viewer ownership

This became the foundational PECS v2 layer.

runtime_topology_engine.py

Purpose:

Canonical runtime topology reconstruction engine.

Effect:

Coordinates deterministic topology reconstruction.

workspace_scanner.py

Purpose:

Scans workspace structure.

Effect:

Provides deterministic workspace discovery.

runtime_entrypoint_mapper.py

Purpose:

Detects runtime entrypoints.

Effect:

Allows runtime topology anchoring.

import_locality_mapper.py

Purpose:

Maps import locality.

Effect:

Provides static locality assistance.

QACTION MODULES

qaction_registry_mapper.py

Purpose:

Detects QAction registrations.

Effect:

Reconstructs QAction continuity.

qaction_ownership_mapper.py

Purpose:

Tracks QAction ownership.

Effect:

Preserves runtime ownership continuity.

menu_topology_mapper.py

Purpose:

Maps menu continuity.

Effect:

Preserves menu dispatch locality.

toolbar_topology_mapper.py

Purpose:

Maps toolbar continuity.

Effect:

Preserves toolbar execution locality.

SIGNAL-SLOT MODULES

signal_slot_mapper.py

Purpose:

Detects signal-slot chains.

Effect:

Reconstructs signal-slot continuity.

signal_chain_builder.py

Purpose:

Builds signal execution chains.

Effect:

Preserves execution-topological continuity.

callback_topology_mapper.py

Purpose:

Maps callback locality.

Effect:

Preserves callback execution continuity.

DISPATCH MODULES

dispatch_chain_mapper.py

Purpose:

Maps runtime dispatch chains.

Effect:

Preserves dispatch-local continuity.

execution_path_builder.py

Purpose:

Builds execution-local paths.

Effect:

Provides deterministic execution locality.

runtime_dispatch_indexer.py

Purpose:

Indexes runtime dispatch locality.

Effect:

Allows dispatch-aware retrieval.

SUBPROCESS MODULES

subprocess_topology_mapper.py

Purpose:

Maps subprocess execution continuity.

Effect:

Preserves subprocess topology reconstruction.

process_chain_builder.py

Purpose:

Builds subprocess execution chains.

Effect:

Provides subprocess-local continuity.

OVERLAY MODULES

overlay_propagation_mapper.py

Purpose:

Maps overlay propagation.

Effect:

Preserves propagation continuity.

state_propagation_tracker.py

Purpose:

Tracks runtime state propagation.

Effect:

Preserves propagation-local execution continuity.

VIEWER MODULES

viewer_ownership_mapper.py

Purpose:

Tracks viewer ownership.

Effect:

Preserves viewer-local continuity.

navigation_propagation_mapper.py

Purpose:

Maps viewer navigation continuity.

Effect:

Preserves navigation-local execution continuity.

EXECUTION GRAPH LAYER

Purpose

Converts runtime reconstruction into:

structured queryable continuity graphs.

runtime_graph.py

Purpose:

Canonical runtime continuity graph.

Effect:

Stores runtime topology structures.

execution_graph.py

Purpose:

Represents execution continuity.

Effect:

Stores execution-local continuity relationships.

ownership_graph.py

Purpose:

Represents ownership continuity.

Effect:

Stores ownership-local execution relationships.

dispatch_graph.py

Purpose:

Represents dispatch continuity.

Effect:

Stores dispatch-local topology.

propagation_graph.py

Purpose:

Represents propagation continuity.

Effect:

Stores overlay and state propagation locality.

graph_consolidator.py

Purpose:

Consolidates runtime continuity graphs.

Effect:

Produces unified queryable continuity structures.

topology_graph_builder.py

Purpose:

Builds topology graph representations.

Effect:

Transforms runtime continuity into structured graph topology.

INDEXING & RETRIEVAL

Purpose

Provides:

- locality indexing

- topology-aware retrieval
- minimal-context reconstruction
- execution-local retrieval

WITHOUT:

- giant workspace scans
 - full-project traversal
-

graph_index.py

Purpose:

Canonical topology graph index.

Effect:

Provides deterministic graph lookup.

execution_index.py

Purpose:

Indexes execution locality.

Effect:

Provides execution-local retrieval.

ownership_index.py

Purpose:

Indexes ownership locality.

Effect:

Provides ownership-aware continuity retrieval.

runtime_path_index.py

Purpose:

Indexes runtime execution paths.

Effect:

Provides runtime-local retrieval.

topology_index.py

Purpose:

Indexes continuity topology.

Effect:

Provides continuity-zone lookup.

locality_index.py

Purpose:

Indexes object locality.

Effect:

Provides locality-directed reconstruction.

graph_query_engine.py

Purpose:

Queries continuity graphs.

Effect:

Provides topology-aware retrieval.

continuity_path_query.py

Purpose:

Queries continuity execution paths.

Effect:

Provides execution-local continuity reconstruction.

topology_retriever.py

Purpose:

Canonical consolidated retrieval authority.

Consolidates:

- locality retrieval
- ownership locality
- execution locality
- continuity reconstruction
- retrieval ranking

Effect:

Provides minimal-context topology-aware continuity retrieval.

This was one of the most important anti-fragmentation consolidations.

SCORING & ARCHAEOLOGY

continuity_score_engine.py

Purpose:

Canonical continuity scoring authority.

Consolidates:

- locality scoring
- runtime scoring
- ownership scoring
- topology scoring

Effect:

Provides deterministic continuity confidence.

continuity_archaeology.py

Purpose:

Canonical continuity archaeology authority.

Tracks:

- duplicate evolution
- canonical drift
- regression-prone topology

Effect:

Provides historical continuity reconstruction.

Important:

archaeology NEVER mutates runtime truth.

INCREMENTAL LAYER

incremental_topology_updater.py

Purpose:

Canonical incremental rebuilding authority.

Consolidates:

- localized invalidation
- selective rebuilding
- locality refresh
- topology refresh
- minimal recomputation

Effect:

Prevents:

- global rescans
- retrieval explosion
- continuity drift
- token inflation

This was another major anti-fragmentation consolidation.

RUNTIME SESSION

workspace_runtime_session.py

Purpose:

Canonical live continuity session.

Tracks:

- active locality
- active topology zones
- engineering focus
- active retrieval state
- continuity cache

Effect:

Provides stabilized live continuity reconstruction.

This became:

the live continuity container.

COMPACTION

compact_context_builder.py

Purpose:

Canonical compact continuity reconstruction authority.

Consolidates:

- compact context generation
- low-token continuity exports
- locality compression
- minimal continuity reconstruction

Effect:

Provides:

- small-model support
- low-token reconstruction
- topology-local compact continuity bundles

This is one of the most practically important PECS modules.

INTEGRATIONS

continue_adapter.py

Purpose:

Provides Continue integration.

Effect:

Generates compact topology-aware continuity context.

copilot_adapter.py

Purpose:

Provides Copilot integration.

Effect:

Provides locality-aware continuity reconstruction.

context_export_adapter.py

Purpose:

Provides compact continuity export generation.

Effect:

Produces low-token continuity bundles.

EXPORTS

continuity_exporter.py

Purpose:

Canonical deterministic export authority.

Effect:

Serializes topology-aware continuity state.

Exports NEVER mutate runtime truth.

VALIDATION

continuity_validator.py

Purpose:

Canonical continuity integrity validator.

Consolidates:

- topology validation
- locality validation
- duplicate authority detection
- continuity integrity validation

Effect:

Provides anti-fragmentation safeguards.

Important:

validation NEVER mutates runtime truth.

RUNTIME ENTRYPOINT

run_pecs_pro.py

Purpose:

Canonical deterministic PECS runtime bootstrap.

Initializes:

- continuity registries
- runtime registries
- graph indexes
- locality indexes
- retrieval infrastructure
- runtime session
- incremental rebuilding
- compact reconstruction

Effect:

Provides one canonical deterministic runtime initialization path.

This prevents:

- distributed startup ownership
 - runtime fragmentation
 - initialization duplication
-

FINAL ARCHITECTURAL STATUS

PECS-PRO v2 is now:

- topology-first
- continuity-oriented
- deterministic
- dense-authority
- low-token
- low-orchestration
- migration-safe
- continuity-compressed

The architecture intentionally avoids:

- duplicate validation paths
- duplicate verification systems
- duplicate consolidators
- orchestration forests
- middleware proliferation
- recursive runtime systems
- AGI abstractions
- autonomous engineering systems

The architecture now focuses on:

minimal-context topology-directed continuity reconstruction.

That is the actual final PECS-PRO v2 objective.